SLOT 13

ERROR / LOADING UI – DETAIL SCREEN VIA API ID

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- FutureBuilder & List Rendering
- Error/Loading UI
- Detail Screen via API ID

ASYNCHRONOUS UI & `FutureBuilder`

Khái niệm quan trọng

Future trong Dart (xử lý bất đồng bộ)
FutureBuilder – widget kết nối Future với UI
`AsyncSnapshot`
Trạng thái: `connectionState`
- waiting
- active
- done

null safety khi xử lý snapshot

Mục tiêu học tập

Hiểu bản chất UI reactive khi làm việc với dữ liệu bất đồng bộ
Biết cách hiển thị Loading / Success / Error đúng chuẩn
Tránh lỗi phổ biến: gọi API nhiều lần khi rebuild

Đây là kỹ thuật cốt lõi trong mọi app có dữ liệu từ server.

LIST RENDERING & DYNAMIC DATA DISPLAY

Khái niệm quan trọng

`ListView.builder`
Efficient rendering (lazy build)
ItemBuilder pattern
Key & identity của widget
Performance với danh sách lớn

Mục tiêu học tập

Hiển thị danh sách dữ liệu từ API
Tối ưu hiệu năng khi render 100+ items
Tránh rebuild không cần thiết

Đây là nền tảng của: news app, e-commerce, social feed, booking app,…

ERROR / LOADING UI DESIGN

Khái niệm quan trọng

Loading indicator (`CircularProgressIndicator`)
Skeleton UI
Empty state
Error state
Retry button

Mục tiêu học tập

Thiết kế UI thân thiện khi:
- Mạng chậm
- API lỗi
- Không có dữ liệu

Hiểu rằng production app ≠ chỉ hiển thị dữ liệu thành công

PhucPT10@fpt.edu.vn

Đây là tư duy chuyên nghiệp khi làm app thực tế.

IV DETAIL SCREEN VIA API ID

◈ Khái niệm quan trọng

Navigation có truyền tham số (ID)
Fetch detail theo ID
Route arguments
Separation giữa List Screen & Detail Screen
Tối ưu: tránh gọi lại API nếu đã có data

🎯 Mục tiêu học tập

Điều hướng từ danh sách → màn hình chi tiết
Hiểu cách xây dựng flow Master–Detail pattern
Thiết kế kiến trúc dữ liệu hợp lý

Đây là pattern phổ biến trong 90% ứng dụng thương mại.

V TƯ DUY KIẾN TRÚC & QUẢN LÝ STATE CƠ BẢN

◈ Khái niệm quan trọng

Không gọi API trực tiếp trong build()
Cache Future
Tách API logic khỏi UI
Phân biệt UI state vs Data state

🎯 Mục tiêu học tập

Viết code sạch, không lặp API
Tránh bug khó phát hiện khi rebuild
Chuẩn bị nền tảng cho Riverpod / Bloc sau này

🎯 TỔNG MỤC TIÊU SLOT 13

Sau bài học, sinh viên phải:

✓ Hiểu cách hiển thị dữ liệu bất đồng bộ đúng chuẩn
✓ Biết render danh sách hiệu quả
✓ Thiết kế đầy đủ Loading / Error / Empty UI
✓ Xây dựng flow List → Detail chuẩn production
✓ Viết code tránh lỗi gọi API lặp

📚 TÀI LIỆU THAM KHẢO UY TÍN

1 Flutter Official Documentation

Website: https://docs.flutter.dev
Tìm kiếm:
- FutureBuilder
- ListView
- Navigation & Routing

👉 Nguồn chính thống, cập nhật mới nhất.

2 Flutter Cookbook (Official)

Phần: Networking & Navigation
Có ví dụ thực tế, dễ áp dụng

3 Reso Coder (YouTube)

Chủ đề: Clean Architecture & FutureBuilder
Phân tích rất sâu về async UI

PhucPT10@fpt.edu.vn

4 FilledStacks (YouTube & Blog)

Chuyên về kiến trúc Flutter production

Phân tích pattern List → Detail chuyên nghiệp

5 Book: “Flutter Complete Reference” – Alberto Miola

Giải thích sâu về asynchronous programming trong Flutter

Phù hợp sinh viên khá – giỏi

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

Future & FutureBuilder (Asynchronous UI)

◈ Khái niệm quan trọng

Future trong Dart (bất đồng bộ, non-blocking)
Cách hoạt động của `async` / `await`
`FutureBuilder`
`AsyncSnapshot`
`connectionState` (waiting, active, done)
null safety khi xử lý snapshot

📌 Sinh viên phải hiểu: UI không chờ API – UI phản ứng theo trạng thái dữ liệu.

◈ Kỹ thuật cần thành thạo

Sử dụng `FutureBuilder` đúng cách
Kiểm tra:

```
snapshot.connectionState
snapshot.hasData
snapshot.hasError
```

Cache Future trong `initState()` để tránh gọi API nhiều lần
Tách hàm gọi API ra khỏi build()

◈ Lỗi thường gặp

❌ Gọi API trực tiếp trong `build()` → gọi lại nhiều lần
❌ Không kiểm tra `hasError`
❌ Truy cập `snapshot.data!` khi chưa có data
❌ Không xử lý trạng thái loading

◈ Best Practices

Luôn xử lý đủ 3 trạng thái: Loading – Success – Error
Không viết logic API trong UI
Tạo biến `late Future<T>` và khởi tạo trong `initState()`
Viết code defensive (kiểm tra null)

List Rendering & Performance

◈ Khái niệm quan trọng

`ListView.builder`
Lazy rendering (chỉ build khi cần)
itemBuilder pattern
Scroll behavior
Key trong danh sách

📌 Sinh viên phải hiểu: Không render toàn bộ danh sách một lần.

◈ Kỹ thuật cần thành thạo

Dùng `ListView.builder` thay vì `Column + map`
Truyền đúng `itemCount`
Tối ưu layout item
Truyền ID khi click item

PhucPT10@fpt.edu.vn

◈ Lỗi thường gặp

- ❌ Dùng `ListView` với danh sách lớn mà không builder
- ❌ Quên set `itemCount`
- ❌ Không xử lý trường hợp danh sách rỗng
- ❌ Nested scroll không kiểm soát

◈ Best Practices

Tách mỗi item thành widget riêng
Thiết kế UI item nhẹ, tối ưu rebuild
Hiển thị Empty State khi list rỗng
Sử dụng const constructor khi có thể

III Error / Loading / Empty UI

◈ Khái niệm quan trọng

Loading state
Error state
Empty state
UX khi mạng chậm

📌 App chuyên nghiệp luôn có trạng thái rõ ràng, không chỉ hiển thị dữ liệu thành công.

◈ Kỹ thuật cần thành thạo

Hiển thị `CircularProgressIndicator`
Hiển thị thông báo lỗi
Thêm nút Retry
Thiết kế Empty UI thân thiện

◈ Lỗi thường gặp

- ❌ Chỉ hiển thị loading
- ❌ Không cho phép retry
- ❌ UI lỗi khó hiểu với người dùng
- ❌ Không phân biệt empty và error

◈ Best Practices

Tách widget Loading riêng
Tách widget Error riêng
Tách widget Empty riêng
Thiết kế UX rõ ràng, thân thiện

IV Navigation & Detail Screen via ID

◈ Khái niệm quan trọng

Navigation truyền tham số
Route arguments
Fetch dữ liệu theo ID
Master–Detail Pattern

📌 Đây là pattern phổ biến nhất trong app thương mại.

◈ Kỹ thuật cần thành thạo

Truyền ID qua Navigator

PhucPT10@fpt.edu.vn

Gọi API theo ID
Xây dựng Detail Screen dùng `FutureBuilder`
Tối ưu tránh gọi lại API nếu đã có dữ liệu

◈ Lỗi thường gặp

❌ Không truyền ID đúng kiểu
❌ Gọi lại API dù đã có data
❌ Không xử lý loading khi mở detail
❌ Không xử lý error riêng cho detail

◈ Best Practices

Chỉ truyền ID thay vì truyền object lớn
Cache dữ liệu nếu cần
Xử lý độc lập Error/Loading ở Detail Screen
Tách riêng List Screen và Detail Screen rõ ràng

☑ Tư duy kiến trúc & Clean Code

◈ Khái niệm quan trọng

Separation of Concerns
UI layer vs Data layer
Single Responsibility
Không trộn logic vào build()

◈ Kỹ năng cần thành thạo

Tách API service
Viết hàm riêng để fetch data
Không lặp code xử lý snapshot
Refactor khi logic quá dài

◈ Lỗi thường gặp

❌ Nhồi toàn bộ code trong một file
❌ Trộn UI và logic
❌ Không tái sử dụng widget

◈ Best Practices

Mỗi màn hình 1 trách nhiệm
Tạo widget nhỏ, dễ test
Tách API service riêng file
Viết code dễ đọc, rõ ràng

🎯 Sau Slot 13, sinh viên BẮT BUỘC phải làm được:

✓ Hiển thị danh sách từ API
✓ Xử lý đầy đủ Loading / Error / Empty
✓ Điều hướng sang Detail bằng ID
✓ Tránh gọi API lặp
✓ Viết code sạch, có cấu trúc

PhucPT10@fpt.edu.vn

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

TÌNH HUỐNG 1: Ứng dụng Thương mại điện tử (E-commerce App)

Bối cảnh

Một công ty xây dựng app bán hàng online.
Màn hình chính hiển thị danh sách sản phẩm lấy từ API server.

Vấn đề gặp phải

API mất 1–3 giây để phản hồi
Người dùng thấy màn hình trắng khi mở app
Khi API lỗi, app crash
Danh sách 200+ sản phẩm gây lag

Cách áp dụng kỹ thuật Slot 13

1 **FutureBuilder**

Dùng `FutureBuilder` để gọi API lấy danh sách sản phẩm
Cache Future trong `initState()` để tránh gọi lại khi rebuild

2 **Loading UI**

Hiển thị `CircularProgressIndicator`
Sau đó cải tiến thành Skeleton UI

3 **Error UI**

Nếu `snapshot.hasError`
    Hiển thị thông báo "Không thể tải sản phẩm"
    Có nút Retry

4 **List Rendering**

Dùng `ListView.builder`
Mỗi item tách thành widget riêng
Chỉ render khi cuộn (lazy loading)

5 **Detail via ID**

Khi click sản phẩm:
    Truyền `productId`
    Gọi API `/products/{id}`
    Hiển thị chi tiết bằng `FutureBuilder`

Mục tiêu đạt được

UX mượt mà
Không crash khi API lỗi
Tối ưu hiệu năng với danh sách lớn
Kiến trúc rõ ràng List → Detail

Bài học rút ra

Không bao giờ hiển thị màn hình trắng
Error handling là bắt buộc
List lớn phải dùng builder
Không gọi API trong build()

TÌNH HUỐNG 2: Ứng dụng Tin tức (News App)

Bối cảnh

App hiển thị danh sách tin tức cập nhật theo ngày.

PhucPT10@fpt.edu.vn

❗ Vấn đề gặp phải

API trả về danh sách rỗng khi không có tin mới
Người dùng không hiểu vì sao màn hình trống
Detail screen load chậm

🛠 Cách áp dụng kỹ thuật

1 Empty State

Nếu danh sách rỗng → hiển thị:
- Icon báo không có tin
- Thông điệp thân thiện

2 FutureBuilder cho từng màn hình

List screen → fetch danh sách
Detail screen → fetch theo articleId

3 Phân tách Loading riêng

Loading List khác Loading Detail
Không block toàn app

🎯 Mục tiêu đạt được

UX chuyên nghiệp
Người dùng hiểu trạng thái hệ thống
Mỗi màn hình tự quản lý trạng thái riêng

📚 Bài học rút ra

Empty state quan trọng như Success state
Detail screen phải xử lý loading riêng
Không phụ thuộc hoàn toàn vào list data

🏢 TÌNH HUỐNG 3: Ứng dụng Ngân hàng (Mobile Banking)

📌 Bối cảnh

App hiển thị danh sách giao dịch của người dùng.

❗ Vấn đề gặp phải

Danh sách giao dịch 500+ item
API timeout khi mạng yếu
Người dùng thoát app khi loading lâu

🛠 Cách áp dụng kỹ thuật

1 Loading State rõ ràng

Hiển thị progress
Sau 5 giây → hiển thị "Mạng chậm"

2 Error Handling chuyên nghiệp

Phân biệt:
- Timeout
- 401
- 500

Hiển thị thông báo khác nhau

3 List Optimization

Dùng `ListView.builder`
Pagination khi cuộn

4 Detail Transaction

Click vào giao dịch → fetch chi tiết theo ID

PhucPT10@fpt.edu.vn

Mục tiêu đạt được

Không lag
Không crash
UX minh bạch khi mạng chậm

Bài học rút ra

Error state phải phân loại
Không load toàn bộ dữ liệu cùng lúc
Performance quan trọng hơn hiệu ứng UI

TÌNH HUỐNG 4: Ứng dụng Đặt lịch khám bệnh

Bối cảnh

Người dùng chọn bác sĩ từ danh sách → xem chi tiết → đặt lịch.

Vấn đề gặp phải

API thay đổi liên tục
Một số bác sĩ không còn lịch trống
App hiển thị dữ liệu cũ

Cách áp dụng

1. **FutureBuilder riêng cho Detail**
   Khi mở màn hình bác sĩ
   Luôn fetch lại theo ID
2. **Hiển thị trạng thái dữ liệu**
   Nếu không có lịch trống → hiển thị Empty state
3. **Tách biệt List & Detail logic**
   List chỉ hiển thị thông tin tóm tắt
   Detail gọi API riêng để đảm bảo chính xác

Mục tiêu đạt được

Dữ liệu chính xác
Tránh hiển thị thông tin cũ
UX rõ ràng

Bài học rút ra

Không phụ thuộc dữ liệu list cho detail
Mỗi màn hình phải độc lập dữ liệu

TỔNG KẾT ỨNG DỤNG THỰC TIỄN

Trong môi trường lập trình chuyên nghiệp, các kỹ thuật Slot 13 được dùng để:

Hiển thị dữ liệu bất đồng bộ
Xây dựng List lớn tối ưu hiệu năng
Thiết kế UI đầy đủ trạng thái
Tách biệt List & Detail theo ID
Tránh crash khi API lỗi
Tạo trải nghiệm người dùng chuyên nghiệp

Điều quan trọng nhất sinh viên phải hiểu

App thực tế luôn có:

Loading

PhucPT10@fpt.edu.vn

Error
Empty
Success

Không bao giờ:

Gọi API trong build()
Hiển thị màn hình trắng
Bỏ qua error handling

Master–Detail Pattern là nền tảng của 80–90% app thương mại.

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

FUTURE & FUTUREBUILDER – ASYNCHRONOUS UI ARCHITECTURE

1 Nền tảng lý thuyết: Asyn chronous Programming trong Dart

Event Loop & Non-blocking Model

Flutter chạy trên một **main isolate (single-thread UI)**.
Nguyên lý:
- UI không được block
- API request không được chạy đồng bộ
- Future đại diện cho một giá trị sẽ có trong tương lai

Dart sử dụng:
- Event loop
- Microtask queue
- Event queue

Khi bạn dùng:

```
await fetchData();
```

Thực chất:
- Hàm tạm dừng
- Event loop tiếp tục xử lý UI
- Khi Future hoàn thành → callback được thực thi

📌 Nếu bạn parse JSON lớn trên main isolate → UI sẽ jank.

2 FutureBuilder hoạt động như thế nào?

FutureBuilder là một **StatefulWidget đặc biệt**:
Nó:
- Nhận một Future
- Lắng nghe trạng thái
- Gọi lại builder khi trạng thái thay đổi

Flow nội bộ:
- Future bắt đầu
- Snapshot = waiting
- Future hoàn thành
- Snapshot = done
- Builder được gọi lại

3 AsyncSnapshot phân tích chuyên sâu

Snapshot gồm:
- connectionState
- data
- error

Các trạng thái:
- none → chưa có future
- waiting → đang chờ
- active → stream
- done → hoàn thành

📌 Trong 90% trường hợp FutureBuilder chỉ dùng waiting & done.

4 Lỗi kiến trúc phổ biến (Production Bug)

❌ Gọi Future trong build()

Sai:

PhucPT10@fpt.edu.vn

```
FutureBuilder(
  future: fetchData(), // BAD
)
```

Vì:

Mỗi rebuild → gọi lại API

Rebuild có thể do:

setState()

MediaQuery change

Orientation change

Đúng:

```
late Future<List<Product>> _future;

initState() {
  _future = fetchData();
}
```

5 Khi nào KHÔNG nên dùng FutureBuilder?

Khi dữ liệu thay đổi liên tục → dùng StreamBuilder

Khi dùng state management (Riverpod, Bloc)

Khi cần caching phức tạp

6 Cập nhật & xu hướng mới

Trong production hiện đại:

FutureBuilder thường được thay bằng:

Riverpod AsyncValue

Bloc state

ValueNotifier

Vì:

Tách UI & logic rõ hơn

Test dễ hơn

Không phụ thuộc lifecycle widget

1 LIST RENDERING – PERFORMANCE & RENDERING PIPELINE

1 Rendering Pipeline trong Flutter

Quá trình vẽ:

Build

Layout

Paint

Compositing

Nếu bạn dùng:

```
Column(
  children: myList.map(...)
)
```

→ Toàn bộ danh sách được build ngay lập tức.

Nếu danh sách 1000 item → crash hoặc lag.

2 ListView.builder – Lazy Rendering

Nguyên lý:

Chỉ build item hiển thị trên màn hình

Khi scroll → build item mới

Item cũ có thể bị dispose

Đây là kỹ thuật cực kỳ quan trọng.

PhucPT10@fpt.edu.vn

3 Memory & GC Implication

Danh sách lớn:

- Nếu không lazy build → RAM tăng mạnh
- Garbage collector hoạt động nhiều
- Frame drop

4 Các chủ điểm nâng cao

- Pagination
- Infinite Scroll
- Pull to refresh
- SliverList
- CustomScrollView

Production app thường:

- Không load 1000 record cùng lúc
- Load theo trang (page 1, page 2…)

5 Keys & Identity

Khi update list:

- Nếu không có key
- Flutter không biết item nào thay đổi

Trong animation list:

```
Key(item.id.toString())
```

Giúp Flutter:

- So sánh widget cũ – mới
- Tránh rebuild sai

I ERROR / LOADING UI – UX ENGINEERING

1 Tư duy chuyên nghiệp

Beginner:

- Chỉ hiển thị dữ liệu

Professional:

- Loading
- Error
- Empty
- Partial data
- Offline state

2 Loading nâng cao

Ngoài CircularProgressIndicator còn có:

- Skeleton loading
- Shimmer effect
- Progressive loading

UX tốt giúp:

- Giảm perceived latency
- Người dùng kiên nhẫn hơn

3 Error phân loại chuyên sâu

Production cần phân biệt:

- 400 → lỗi người dùng
- 401 → token hết hạn

PhucPT10@fpt.edu.vn

403 → không quyền
404 → không tìm thấy
500 → lỗi server
Timeout
No Internet

Không nên hiển thị chung “Something went wrong”.

4 Retry Strategy

Professional app:

Có nút retry
Hoặc auto retry
Hoặc exponential backoff

5 Offline-first Pattern

App lớn:

Cache data local
Nếu offline → hiển thị cache
Khi online → sync lại

IV DETAIL SCREEN VIA API ID – MASTER DETAIL PATTERN

1 Kiến trúc Master–Detail

Pattern:

List screen (summary data)
Detail screen (full data)

Không nên:

Truyền object lớn qua Navigator
Dựa hoàn toàn vào list data

2 Vì sao phải gọi API theo ID?

Vì:

List data có thể thiếu field
Dữ liệu có thể đã thay đổi
Bảo mật

3 Tối ưu nâng cao

Professional app thường:

Cache detail nếu đã fetch
Sử dụng repository layer
Đồng bộ dữ liệu giữa list và detail

4 Navigation nâng cao

Các cách truyền ID:

Constructor
Named route
onGenerateRoute

Production app thường dùng:

Centralized route management

V CÁC VẤN ĐỀ THỰC TIỄN LỚN TRONG DỰ ÁN

PhucPT10@fpt.edu.vn

1 UI Jank khi parse JSON lớn

Giải pháp:

- compute()
- isolate
- pagination

2 Race Condition khi mở detail liên tục

Nếu user bấm 3 item nhanh:

- 3 API chạy song song
- Response về sai thứ tự

Giải pháp:

- Cancel request
- Debounce navigation

3 Memory Leak do giữ reference Future

Khi màn hình dispose:

- Future vẫn chạy
- setState sau dispose → crash

Giải pháp:

- Check mounted

🎯 Kiến thức đầy đủ bạn phải đạt được sau Slot 13

Bạn không chỉ biết:

- Cách viết FutureBuilder

Mà phải hiểu:

- Event loop
- Rendering pipeline
- Memory impact
- UX state design
- Kiến trúc Master–Detail
- Khi nào nâng cấp lên state management

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

FUTUREBUILDER – KINH NGHIỆM THỰC CHIẾN

Kinh nghiệm thực tế

Trong dự án thật, vấn đề phổ biến nhất không phải là "không biết dùng FutureBuilder"
Mà là:
Future bị gọi lại nhiều lần ngoài ý muốn.

Tình huống thực tế

App thương mại điện tử:

- Mở màn hình danh sách sản phẩm
- Người dùng xoay màn hình
- API bị gọi lại
- Server load tăng gấp đôi

Nguyên nhân: Gọi API trực tiếp trong build()

Lỗi thường gặp

Lỗi 1: API bị gọi lại nhiều lần

Sai:

```
FutureBuilder(
  future: fetchProducts(),
)
```

Nguyên nhân:

- build() có thể chạy lại nhiều lần

Cách khắc phục:

```
late Future<List<Product>> _future;

initState() {
  _future = fetchProducts();
}
```

Lỗi 2: setState() sau khi dispose

Tình huống:

- Người dùng bấm back khi đang loading
- Future hoàn thành
- setState() được gọi
- App crash

Nguyên nhân:
Widget đã dispose.

Khắc phục:

```
if (!mounted) return;
```

Tips thực chiến

- Log connectionState để debug nhanh
- Tách LoadingWidget / ErrorWidget thành component riêng
- Luôn test trường hợp mạng chậm bằng cách delay giả lập

Best Practices

- Không bao giờ gọi API trong build()
- Không viết logic fetch trực tiếp trong widget nếu dự án lớn
- Luôn xử lý 3 trạng thái: loading – error – success
- Tách data layer khỏi UI layer

PhucPT10@fpt.edu.vn

🔄 Liên hệ thực tế

Các app lớn như app thương mại, ngân hàng:

Không dùng FutureBuilder đơn thuần
Dùng state management (Riverpod / Bloc)
Nhưng nguyên lý async UI vẫn giống 100%

📋 LIST RENDERING – PERFORMANCE & MEMORY

✅ Kinh nghiệm thực tế

Danh sách 100 item chạy tốt trên máy test
Nhưng 2000 item trên máy cũ → lag.
Sinh viên thường không test với data lớn.

⚠️ Lỗi thường gặp

❌ Lỗi 1: Dùng Column thay vì ListView.builder

Sai:

```
Column(
  children: items.map(...)
)
```

Nguyên nhân:
Render toàn bộ list ngay lập tức.

✅ Khắc phục:
Dùng ListView.builder.

❌ Lỗi 2: Nested Scroll không kiểm soát

Ví dụ:
ListView bên trong SingleChildScrollView
→ Render không đúng / crash layout.

❌ Lỗi 3: Không xử lý danh sách rỗng

UI trắng → người dùng nghĩ app lỗi.

💡 Tips thực chiến

Tách mỗi item thành StatelessWidget riêng
Dùng const constructor khi có thể
Test với mock data 1000 record
Dùng Flutter DevTools để check rebuild

🛡️ Best Practices

Luôn dùng ListView.builder cho list động
Nếu data lớn → pagination
Nếu scroll phức tạp → dùng CustomScrollView + SliverList
Không render image full resolution trong list

🔄 Liên hệ thực tế

App mạng xã hội:

Không bao giờ load 1000 post một lúc
Luôn có infinite scroll + pagination

⚠️ ERROR / LOADING UI – UX CHUYÊN NGHIỆP

PhucPT10@fpt.edu.vn

✅ Kinh nghiệm thực tế

Người dùng ghét:

- Màn hình trắng
- Thông báo lỗi chung chung
- Không có nút retry

Loading UX ảnh hưởng trực tiếp đến retention rate.

⚠ Lỗi thường gặp

❌ Lỗi 1: Chỉ hiển thị CircularProgressIndicator

UX nghèo nàn.

❌ Lỗi 2: Không phân biệt empty và error

Empty ≠ Error

❌ Lỗi 3: Không xử lý timeout

Mạng yếu → loading vô hạn.

💡 Tips thực chiến

- Thêm timeout thủ công cho API
- Thiết kế UI loading giống layout thật (skeleton)
- Log error ra console để debug

🛡 Best Practices

- Phân biệt:
  - No internet
  - Server error
  - Empty data
- Có nút Retry
- Không hiển thị raw error message cho user
- Không expose stack trace

🔄 Liên hệ thực tế

App ngân hàng:

- Nếu 401 → tự logout
- Nếu timeout → hiển thị mạng yếu
- Không bao giờ hiển thị thông báo kỹ thuật

IV. DETAIL SCREEN VIA API ID – MASTER DETAIL

✅ Kinh nghiệm thực tế

Sinh viên thường truyền toàn bộ object qua Navigator.
Nhưng trong dự án thật:

- Object có thể 200KB
- Data có thể cũ

⚠ Lỗi thường gặp

❌ Lỗi 1: Không gọi lại API detail

Hậu quả:
Hiển thị dữ liệu cũ.

❌ Lỗi 2: Race condition

User bấm nhanh 3 item → 3 API gọi song song.

PhucPT10@fpt.edu.vn

Response về sai thứ tự → hiển thị sai dữ liệu.

Tips thực chiến

Chỉ truyền ID
Cache detail nếu cần
Disable button khi đang loading

Best Practices

List chỉ hiển thị summary
Detail gọi API riêng
Nếu cần hiệu năng → preload trước
Tách Repository layer

Liên hệ thực tế

App đặt phòng khách sạn:
List hiển thị tên & giá
Detail gọi API riêng để đảm bảo còn phòng

TỔNG KẾT NHỮNG ĐIỀU SINH VIÊN CẦN NHỚ

5 lỗi phổ biến nhất trong Slot 13

Gọi API trong build()
Không xử lý error
Dùng Column cho list lớn
Không xử lý empty state
Truyền object lớn qua Navigator

5 nguyên tắc production-level

UI không bao giờ trắng
Error phải được phân loại
List phải lazy load
Không phụ thuộc dữ liệu list cho detail
Không trộn logic và UI

.

PhucPT10@fpt.edu.vn

EXERCISE

📄 BÀI TẬP TỔNG HỢP

Product Explorer App (Master–Detail API App)

I) TÓM TẮT DỰ ÁN (Product Explorer App)

📌 Tên dự án

Product Explorer App (Master–Detail API App)

🎯 Mục tiêu học tập

Sau bài này, sinh viên sẽ làm được:

- Lấy dữ liệu từ API bằng `http`
- Render danh sách bằng `ListView.builder`
- Dùng `FutureBuilder` đúng cách: **Loading / Error / Empty / Data**
- **Cache Future** để tránh gọi API lặp
- Điều hướng Master → Detail bằng **ID**
- Màn hình Detail gọi API riêng theo ID (có Loading/Error riêng)
- Biết thêm **Retry** khi lỗi mạng

II) CHỨC NĂNG CHÍNH

- Fetch danh sách sản phẩm từ API: `GET /products`
- Hiển thị danh sách bằng `ListView.builder`
- Loading UI (widget riêng)
- Error UI + Retry (widget riêng)
- Tap item → điều hướng sang Detail
- Fetch chi tiết theo ID: `GET /products/{id}`
- Loading/Error riêng cho Detail
- Tránh gọi API lặp: **không gọi API trong `build()`**

III) CÔNG NGHỆ SỬ DỤNG

- Flutter (Stable), Dart
- `async/await`
- `FutureBuilder`
- `Navigator` / `MaterialPageRoute`
- `http` package
- API:
  - https://fakestoreapi.com/products
  - https://fakestoreapi.com/products/{id}

IV) CẤU TRÚC THƯ MỤC

Text

```
lib/
 ├── main.dart
 ├── models/
 │    └── product.dart
 ├── services/
 │    └── api_service.dart
 ├── screens/
 │    ├── product_list_screen.dart
 │    └── product_detail_screen.dart
```


PhucPT10@fpt.edu.vn

```
└── widgets/
    ├── loading_widget.dart
    └── error_widget.dart
```

V) TÓM TẮT DANH SÁCH BƯỚC THỰC HIỆN (Checklist)

- Tạo project Flutter
- Thêm package `http`
- Tạo model `Product`
- Tạo `ApiService` gọi API
- Làm UI Loading/Error dạng widget tái sử dụng
- Làm Product List Screen:
  - cache Future trong `initState`
  - FutureBuilder + ListView.builder
  - tap để mở detail theo `id`
- Làm Product Detail Screen:
  - nhận `productId`
  - gọi API theo ID (cache Future)
  - FutureBuilder riêng
- Test đủ case: Loading, Error (tắt mạng), Empty (mô phỏng), Back/Forward

VI) HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (rất "cầm tay chỉ việc")

BƯỚC 1: TẠO PROJECT

Thao tác

- Mở terminal:
  ```
  flutter create product_explorer
  cd product_explorer
  ```
- Mở VSCode:
  ```
  code .
  ```

Kiểm chứng

- Chạy `flutter run` thấy app mẫu chạy được.

BƯỚC 2: THÊM HTTP PACKAGE

Thao tác

- Mở `pubspec.yaml`
- Trong `dependencies:` thêm:

```yaml
http: ^0.13.6
```

- Chạy:
  `flutter pub get`

Kiểm chứng

- Không báo lỗi đỏ.
- File `.dart_tool` và `pubspec.lock` được cập nhật.

BƯỚC 3: TẠO PRODUCT MODEL

Mục tiêu: Parse JSON → Dart object.

✅ **Kiểm chứng kỹ thuật**: mở API trên trình duyệt, nhìn key `id/title/price/description/image` và đối chiếu trong model.

PhucPT10@fpt.edu.vn

lib/models/product.dart

```
class Product {
  final int id;
  final String title;
  final String description;
  final String image;
  final double price;

  Product({
    required this.id,
    required this.title,
    required this.description,
    required this.image,
    required this.price,
  });

  factory Product.fromJson(Map<String, dynamic> json) {
    return Product(
      id: json['id'] as int,
      title: json['title'] as String,
      description: json['description'] as String,
      image: json['image'] as String,
      price: (json['price'] as num).toDouble(),
    );
  }
};
```

BƯỚC 4: TẠO API SERVICE (LIST + DETAIL)

Mục tiêu: tách code gọi API ra khỏi UI.

✅ **Kiểm chứng kỹ thuật**:

Nếu đổi URL sai → sẽ vào nhánh throw Exception.

Có thể `print(response.statusCode)` để thấy 200.

lib/services/api_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

import '../models/product.dart';

class ApiService {
  static const String baseUrl = "https://fakestoreapi.com/products";

  static Future<List<Product>> fetchProducts() async {
    final response = await http.get(Uri.parse(baseUrl));

    if (response.statusCode == 200) {
      final List data = json.decode(response.body) as List;
      return data.map((e) => Product.fromJson(e as Map<String,
dynamic>)).toList();
    } else {
      throw Exception("Failed to load products (status:
${response.statusCode})");
    }
```


PhucPT10@fpt.edu.vn

```
  }

  static Future<Product> fetchProductById(int id) async {
    final response = await http.get(Uri.parse("$baseUrl/$id"));

    if (response.statusCode == 200) {
      return Product.fromJson(json.decode(response.body) as Map<String,
dynamic>);
    } else {
      throw Exception("Failed to load product detail (status:
${response.statusCode})");
    }
  }
} {
```

BƯỚC 5: TẠO WIDGET LOADING + ERROR (TÁI SỬ DỤNG)
Lý do: sinh viên yếu hay copy/paste Loading/Error lung tung → khó sửa đồng loạt. Ta gom thành widget.

5.1 LoadingWidget
lib/widgets/loading_widget.dart

```
import 'package:flutter/material.dart';

class LoadingWidget extends StatelessWidget {
  final String message;

  const LoadingWidget({super.key, this.message = "Loading..."});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          const CircularProgressIndicator(),
          const SizedBox(height: 12),
          Text(message),
        ],
      ),
    );
  }
}
```

5.2 AppErrorWidget (có Retry)
lib/widgets/error_widget.dart

```
import 'package:flutter/material.dart';

class AppErrorWidget extends StatelessWidget {
  final String message;
  final VoidCallback onRetry;
```


PhucPT10@fpt.edu.vn

```
  const AppErrorWidget({
    super.key,
    required this.message,
    required this.onRetry,
  });

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(
              message,
              textAlign: TextAlign.center,
              style: const TextStyle(color: Colors.red),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: onRetry,
              child: const Text("Retry"),
            ),
          ],
        ),
      ),
    );
  }
}
```

✅ **Kiểm chứng kỹ thuật**:
Tắt mạng → app hiện Error UI → bấm Retry (sau khi bật lại mạng) → load lại.

VII) FUTUREBUILDER & LIST RENDERING (MASTER SCREEN)
Tư duy quan trọng cho sinh viên yếu
`build()` có thể chạy **rất nhiều lần**.
Nếu gọi `ApiService.fetchProducts()` trong `build()` → mỗi lần build là gọi API lại.
Giải pháp: **cache Future** trong `initState()` bằng biến `_future`.

lib/screens/product_list_screen.dart

```
import 'package:flutter/material.dart';

import '../models/product.dart';
import '../services/api_service.dart';
import '../widgets/error_widget.dart';
import '../widgets/loading_widget.dart';
import 'product_detail_screen.dart';

class ProductListScreen extends StatefulWidget {
  const ProductListScreen({super.key});
```


PhucPT10@fpt.edu.vn

```
  @override
  State<ProductListScreen> createState() => _ProductListScreenState();
}

class _ProductListScreenState extends State<ProductListScreen> {
  late Future<List<Product>> _future;

  @override
  void initState() {
    super.initState();
    _future = ApiService.fetchProducts(); // cache Future ở đây
  }

  void _retry() {
    setState(() {
      _future = ApiService.fetchProducts();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Products")),
      body: FutureBuilder<List<Product>>(
        future: _future,
        builder: (context, snapshot) {
          // 1) Loading
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const LoadingWidget(message: "Loading products...");
          }

          // 2) Error
          if (snapshot.hasError) {
            return AppErrorWidget(
              message: "Error loading products.\n${snapshot.error}",
              onRetry: _retry,
            );
          }

          // 3) Empty / No data
          final products = snapshot.data;
          if (products == null || products.isEmpty) {
            return const Center(child: Text("No products found"));
          }

          // 4) Data UI
          return ListView.builder(
            itemCount: products.length,
            itemBuilder: (context, index) {
              final product = products[index];

              return ListTile(
```


PhucPT10@fpt.edu.vn

```
                leading: Image.network(
                  product.image,
                  width: 48,
                  height: 48,
                  fit: BoxFit.cover,
                  errorBuilder: (_, __, ___) => const
Icon(Icons.image_not_supported),
                ),
                title: Text(
                  product.title,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                ),
                subtitle: Text("\$${product.price}"),
                onTap: () {
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (_) => ProductDetailScreen(productId:
product.id),
                    ),
                  );
                },
              );
            },
          );
        },
      ),
    );
  }
}
```

✅ Cách kiểm chứng "FutureBuilder & List Rendering"

Chạy app → thấy "Loading products..."

Sau 1–2 giây → list hiện ra

Cuộn list mượt (ListView.builder chỉ build item khi cần)

Quan sát debug console (tuỳ bạn thêm print trong service) để thấy API không bị gọi lại khi scroll.

VIII) DETAIL SCREEN VIA API ID (TRỌNG TÂM)

Mục tiêu: màn Detail nhận `productId`, gọi API `/products/{id}`.

Điểm dạy quan trọng

Detail cũng phải **cache Future** trong `initState()`

Nếu không cache, mỗi lần setState/build lại có thể gọi lại API

lib/screens/product_detail_screen.dart

```
import 'package:flutter/material.dart';

import '../models/product.dart';
import '../services/api_service.dart';
import '../widgets/error_widget.dart';
import '../widgets/loading_widget.dart';
```


PhucPT10@fpt.edu.vn

```
class ProductDetailScreen extends StatefulWidget {
  final int productId;

  const ProductDetailScreen({super.key, required this.productId});

  @override
  State<ProductDetailScreen> createState() => _ProductDetailScreenState();
}

class _ProductDetailScreenState extends State<ProductDetailScreen> {
  late Future<Product> _future;

  @override
  void initState() {
    super.initState();
    _future = ApiService.fetchProductById(widget.productId);
  }

  void _retry() {
    setState(() {
      _future = ApiService.fetchProductById(widget.productId);
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Product Detail")),
      body: FutureBuilder<Product>(
        future: _future,
        builder: (context, snapshot) {
          // Loading
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const LoadingWidget(message: "Loading detail...");
          }

          // Error
          if (snapshot.hasError) {
            return AppErrorWidget(
              message: "Error loading detail.\n${snapshot.error}",
              onRetry: _retry,
            );
          }

          // No data
          final product = snapshot.data;
          if (product == null) {
            return const Center(child: Text("No data"));
          }

          // Data UI
          return SingleChildScrollView(
```


PhucPT10@fpt.edu.vn

```
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Center(
                  child: Image.network(
                    product.image,
                    height: 200,
                    fit: BoxFit.contain,
                    errorBuilder: (_, __, ___) => const
Icon(Icons.image_not_supported, size: 80),
                  ),
                ),
                const SizedBox(height: 16),
                Text(
                  product.title,
                  style: const TextStyle(fontSize: 20, fontWeight:
FontWeight.bold),
                ),
                const SizedBox(height: 8),
                Text("\$${product.price}", style: const TextStyle(fontSize:
18)),
                const SizedBox(height: 16),
                Text(product.description),
              ],
            ),
          );
        },
      ),
    );
  }
}
```

✅ Cách kiểm chứng "Detail via API ID"

Bấm 1 sản phẩm bất kỳ

Màn detail hiện "Loading detail..."

Hiện đúng title/price/description khác với item khác

Back về list không crash

Tắt mạng khi đang ở detail → hiện error + Retry

IX) MAIN.DART

lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/product_list_screen.dart';

void main() {
  runApp(const ProductExplorerApp());
}

class ProductExplorerApp extends StatelessWidget {
  const ProductExplorerApp({super.key});
```


PhucPT10@fpt.edu.vn

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Product Explorer',
      theme: ThemeData(useMaterial3: true),
      home: const ProductListScreen(),
    );
  }
}
```

X) TỔNG HỢP ĐẦY ĐỦ CODE (TẤT CẢ FILE)

Bạn đã có đầy đủ code của:

- `lib/main.dart`
- `lib/models/product.dart`
- `lib/services/api_service.dart`
- `lib/screens/product_list_screen.dart`
- `lib/screens/product_detail_screen.dart`
- `lib/widgets/loading_widget.dart`
- `lib/widgets/error_widget.dart`

(ở các file block phía trên)

XI) CÁC "PHIÊN BẢN" THEO DIỄN BIẾN THỰC HIỆN (V1 → V5)

Mục tiêu phần này: bạn dạy theo từng mốc, sinh viên copy theo từng phiên bản, dễ theo dõi "tại sao cần bước này".

V1 — App chạy được (hello UI)

Chỉ có `main.dart` + màn hình tĩnh.

V2 — Có Model + ApiService

Thêm `product.dart`, `api_service.dart`

Chưa có UI list.

V3 — List Screen bằng FutureBuilder (chưa tách widget Loading/Error)

`ProductListScreen` dùng `FutureBuilder`

Loading/Error hiển thị trực tiếp bằng `Center(Text...)` + `CircularProgressIndicator`.

V4 ??? Tách LoadingWidget + AppErrorWidget + Retry

Thêm `widgets/loading_widget.dart` và `widgets/error_widget.dart`

List Screen dùng Retry để gọi lại `_future`.

V5 — Detail Screen via API ID (hoàn chỉnh Master–Detail)

Thêm `ProductDetailScreen`

List → Detail theo `product.id`

Detail có Loading/Error + Retry riêng

XII) KIỂM CHỨNG TỪNG KỸ THUẬT (Checklist chấm điểm)

FutureBuilder hoạt động

mở app thấy loading

có data thì hiện list

List rendering chuẩn

list cuộn mượt

item hiển thị title/price

Error UI

PhucPT10@fpt.edu.vn

tắt mạng → hiện error
bật mạng → bấm Retry → load lại OK

Detail theo ID

bấm item A → detail A
bấm item B → detail B khác A

Không gọi API lặp

không đặt `fetchProducts()` trong `build()`
future được tạo trong `initState()`

XIII) LỖI THƯỜNG GẶP & CÁCH SỬA (dành cho SV yếu)

1) API bị gọi lại liên tục

Nguyên nhân: gọi `ApiService.fetchProducts()` trong `build()`

Cách sửa: cache Future như:

```
late Future<List<Product>> _future;
```

gán trong `initState()`

2) `snapshot.data` null nhưng vẫn dùng

Sửa: luôn kiểm tra:

`if (!snapshot.hasData) ...` hoặc `final data = snapshot.data; if (data == null) ...`

3) Ảnh không hiện

Test `errorBuilder`

Android cần internet permission (Flutter thường đã có, nhưng nếu gặp lỗi thì kiểm tra `AndroidManifest.xml`).

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu	Ứng dụng trong dự án
API (Application Programming Interface)	Giao diện cho phép các hệ thống giao tiếp với nhau	App Flutter gọi server để lấy dữ liệu	Gọi API để lấy danh sách sản phẩm, user
REST API	Kiến trúc API dựa trên HTTP	Dùng GET/POST/PUT/DELETE	Xây dựng backend cho app mobile
Endpoint	Địa chỉ URL cụ thể của API	Ví dụ: `/users/5`	Lấy user theo ID
API ID	Định danh duy nhất của một tài nguyên	Số ID để truy vấn dữ liệu cụ thể	Lấy chi tiết sản phẩm theo ID
HTTP Method	Phương thức gửi request	GET, POST, PUT, DELETE	CRUD dữ liệu
GET	Lấy dữ liệu từ server	Không thay đổi dữ liệu	Hiển thị danh sách
POST	Tạo mới dữ liệu	Gửi dữ liệu lên server	Tạo user mới
PUT	Cập nhật toàn bộ dữ liệu	Thay thế dữ liệu cũ	Update profile
PATCH	Cập nhật một phần	Update một field	Update avatar
DELETE	Xóa dữ liệu	Gửi ID cần xóa	Xóa sản phẩm
Request	Yêu cầu gửi từ client	App gửi yêu cầu lên server	Lấy danh sách bài viết
Response	Phản hồi từ server	Server trả JSON về	Nhận dữ liệu user
Status Code	Mã trạng thái HTTP	200, 404, 500	Kiểm tra thành công hay lỗi
200 OK	Request thành công	Có dữ liệu trả về	Hiển thị UI
201 Created	Tạo thành công	Sau POST	Hiển thị thông báo thành công
400 Bad Request	Request sai	Thiếu tham số	Báo lỗi validate
401 Unauthorized	Chưa đăng nhập	Thiếu token	Redirect login
404 Not Found	Không tìm thấy	Sai ID	Hiển thị "Không tồn tại"
500 Internal Server Error	Lỗi server	Lỗi backend	Hiển thị thông báo lỗi
JSON	Định dạng dữ liệu trao đổi	Dạng key-value	Parse sang Model
Serialization	Chuyển object → JSON	Gửi dữ liệu lên server	Khi POST
Deserialization	JSON → Object	Nhận dữ liệu từ server	Parse Model
Model	Class đại diện dữ liệu	Ví dụ: UserModel	Map JSON thành object
fromJson()	Hàm parse JSON	Tạo object từ JSON	Tạo instance User
toJson()	Chuyển object thành JSON	Gửi dữ liệu đi	Khi gọi POST
Async / Await	Lập trình bất đồng bộ	Chờ API trả dữ liệu	fetchData() async
Future	Giá trị sẽ có trong tương lai	API trả về Future	Future<User>
FutureBuilder	Widget xử lý Future	Hiển thị loading/data/error	Giao diện chờ API
try-catch	Bắt lỗi runtime	Tránh crash app	Bắt lỗi network
Exception	Ngoại lệ	Lỗi khi gọi API	Timeout
Timeout	Hết thời gian chờ	Server phản hồi chậm	Cần xử lý fallback

PhucPT10@fpt.edu.vn

Dio	Thư viện HTTP nâng cao	Có interceptor	Quản lý token
http package	Thư viện HTTP cơ bản	Dễ dùng	Gọi GET đơn giản
Interceptor	Chặn request/response	Thêm token tự động	Gắn Authorization
Authorization Header	Header chứa token	Bearer token	API cần xác thực
Token	Chuỗi xác thực	JWT token	Login thành công
JWT	JSON Web Token	Token phổ biến	Lưu session
CRUD	Create, Read, Update, Delete	4 thao tác dữ liệu	Xây app quản lý
Repository Pattern	Tách logic dữ liệu	Clean Architecture	Tổ chức code
Service Layer	Lớp xử lý API	Tách khỏi UI	user_service.dart
Clean Architecture	Kiến trúc tách lớp	UI – Domain – Data	Dự án lớn
MVC	Model-View-Controller	Tách logic	Dự án nhỏ
State Management	Quản lý trạng thái	Provider, Bloc	Quản lý dữ liệu API
Provider	State management đơn giản	Dễ học	Lưu danh sách API
Bloc	State management nâng cao	Event → State	App lớn
Loading State	Trạng thái đang tải	Hiển thị spinner	Khi gọi API
Error State	Trạng thái lỗi	Hiển thị thông báo	API fail
Empty State	Không có dữ liệu	UI riêng	Danh sách rỗng
Pagination	Phân trang	Load từng phần	Danh sách dài
Infinite Scroll	Cuộn tải thêm	Tối ưu UX	Facebook feed
Debounce	Trì hoãn request	Tránh spam API	Search
Caching	Lưu tạm dữ liệu	Giảm gọi API	Lưu danh sách
Secure Storage	Lưu trữ an toàn	Lưu token	flutter_secure_storage
Environment Config	Cấu hình môi trường	Dev / Production	Đổi baseUrl
Base URL	Đường dẫn gốc API	https://api.domain.com	Cấu hình chung
API Versioning	Quản lý phiên bản API	/v1/users	Tránh lỗi update
Mock API	API giả lập	Test nhanh	jsonplaceholder
Postman	Công cụ test API	Kiểm tra response	Debug backend
Swagger	Tài liệu API	Mô tả endpoint	Hiểu cách gọi
Rate Limit	Giới hạn request	Tránh spam	API public
Throttling	Giới hạn tốc độ	Bảo vệ server	Ứng dụng lớn
Network Layer	Tầng xử lý mạng	Chứa API call	api_service.dart
DTO (Data Transfer Object)	Object truyền dữ liệu	Dùng trung gian	Map Model
Domain Layer	Lớp nghiệp vụ	Business logic	Validation
Data Layer	Lớp dữ liệu	API, Database	Gọi server

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

1 Phân tích kiến trúc API trong ứng dụng lớn

Phân tích sự khác biệt giữa kiến trúc gọi API trực tiếp trong Widget và việc sử dụng Repository Pattern trong Flutter.
Đánh giá ưu – nhược điểm của từng cách trong dự án có quy mô 50+ API endpoints.
Đề xuất kiến trúc tối ưu cho một ứng dụng thương mại điện tử.
🎯 Mục tiêu: hiểu sâu Clean Architecture & maintainability.

2 Đánh giá chiến lược xử lý lỗi API

Phân tích các cấp độ lỗi khi gọi API (Network error, HTTP error, Business logic error).
Đánh giá cách tổ chức Exception handling tập trung trong Flutter.
Thiết kế một hệ thống xử lý lỗi có thể tái sử dụng cho toàn bộ dự án.
🎯 Mục tiêu: nâng cao tư duy error management chuyên nghiệp.

3 Phân tích hiệu suất khi gọi API nhiều lần

Giả sử một màn hình cần gọi 5 API song song.
Phân tích tác động đến performance và UX.
So sánh cách dùng `Future.wait()` và gọi tuần tự.
Đề xuất giải pháp tối ưu.
🎯 Mục tiêu: hiểu async concurrency và tối ưu trải nghiệm.

4 Đánh giá bảo mật khi làm việc với API ID

Phân tích các rủi ro bảo mật khi sử dụng API ID trực tiếp từ client.
Tại sao không nên tin tưởng ID từ phía client?
Đề xuất cơ chế bảo vệ dữ liệu (JWT, role-based access control, backend validation).
🎯 Mục tiêu: hiểu client-server trust boundary.

5 So sánh REST API và GraphQL trong Flutter

Phân tích sự khác biệt giữa REST và GraphQL khi xây dựng app Flutter có nhiều màn hình chi tiết theo ID.
Khi nào GraphQL mang lại lợi ích vượt trội?
Đánh giá độ phức tạp triển khai.
🎯 Mục tiêu: tư duy kiến trúc API hiện đại.

6 Thiết kế chiến lược caching thông minh

Phân tích các kỹ thuật caching dữ liệu API ID (Memory cache, Local DB, Hive, SharedPreferences).
Đánh giá khi nào nên cache và khi nào không nên.
Thiết kế chiến lược cache cho ứng dụng tin tức.
🎯 Mục tiêu: tối ưu performance và giảm server load.

7 Phân tích Pagination & Infinite Scroll

So sánh offset pagination và cursor pagination.
Đánh giá tác động của từng loại trong Flutter ListView với API ID.
Thiết kế giải pháp infinite scroll cho ứng dụng mạng xã hội.
🎯 Mục tiêu: hiểu sâu xử lý dữ liệu lớn.

PhucPT10@fpt.edu.vn

8 Đánh giá State Management trong xử lý API

So sánh Provider, Riverpod và Bloc khi xử lý API có nhiều trạng thái (Loading, Success, Error, Refresh).
Phân tích mức độ phức tạp khi mở rộng dự án.
Đề xuất giải pháp phù hợp cho app enterprise.
🎯 Mục tiêu: nâng cấp tư duy tổ chức state chuyên nghiệp.

9 Phân tích chiến lược Retry & Timeout

Khi API bị timeout hoặc mất mạng tạm thời, ứng dụng nên xử lý như thế nào?
Phân tích kỹ thuật retry với exponential backoff.
Thiết kế cơ chế retry không gây spam server.
🎯 Mục tiêu: xử lý tình huống thực tế production.

10 Thiết kế hệ thống API Layer chuẩn Production

Thiết kế một API Layer cho ứng dụng Flutter có:

- Interceptor
- Token auto refresh
- Logging
- Error mapping
- Environment config (dev/staging/prod)

Phân tích cách tổ chức thư mục và dependency injection để đảm bảo dễ maintain.
🎯 Mục tiêu: tư duy ở mức Senior Developer.
.

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

1 Khi Backend thay đổi API giữa chừng
Trong dự án thực tế, backend thay đổi cấu trúc JSON của API ID mà không báo trước.
Phân tích hậu quả với Flutter app đã release.
Thiết kế cơ chế versioning và backward compatibility để tránh crash production.
🎯 Tư duy: quản lý thay đổi & ổn định hệ thống.

2 API chậm trên thiết bị thật nhưng nhanh trên giả lập
Phân tích nguyên nhân khiến API ID load nhanh trên emulator nhưng chậm trên thiết bị thật (3G/4G).
Đề xuất giải pháp tối ưu UX khi mạng yếu.
🎯 Tư duy: network latency & UX thực tế.

3 Quản lý Token hết hạn trong ứng dụng lớn
Thiết kế cơ chế auto refresh JWT token bằng interceptor.
Phân tích race condition khi nhiều request đồng thời bị 401.
🎯 Tư duy: concurrency & security.

4 Khi API ID bị lộ logic nghiệp vụ
Phân tích rủi ro khi client có thể tự thay đổi ID để truy cập dữ liệu người khác.
Đề xuất chiến lược bảo mật backend để tránh IDOR (Insecure Direct Object Reference).
🎯 Tư duy: bảo mật production.

5 Đồng bộ dữ liệu khi Offline → Online
Thiết kế chiến lược đồng bộ dữ liệu khi user chỉnh sửa dữ liệu offline rồi online trở lại.
Phân tích conflict resolution.
🎯 Tư duy: offline-first architecture.

6 Tối ưu List lớn 10.000 phần tử
Phân tích tác động của việc load danh sách lớn qua API ID.
So sánh pagination, lazy loading, caching.
Đề xuất giải pháp cho app thương mại điện tử.
🎯 Tư duy: performance & scalability.

7 Logging & Monitoring API Production
Thiết kế hệ thống logging request/response trong môi trường production.
Phân tích cách sử dụng các công cụ như
Firebase Crashlytics hoặc
Sentry để theo dõi lỗi API.
🎯 Tư duy: observability & maintainability.

8 Refactor dự án nhỏ thành Clean Architecture
Một dự án Flutter ban đầu gọi API trực tiếp trong Widget.
Phân tích khó khăn khi scale lên 30 màn hình.
Thiết kế lộ trình refactor sang Repository Pattern.
🎯 Tư duy: technical debt & refactoring.

9 Thiết kế API Layer dùng Dependency Injection
Phân tích lợi ích của việc dùng DI (GetIt hoặc Riverpod) cho API Service.
Đánh giá khả năng test và mở rộng hệ thống.

PhucPT10@fpt.edu.vn

🎯 Tư duy: testability & modularization.

10 Quản lý môi trường Dev – Staging – Production
Thiết kế cấu hình baseURL động cho nhiều môi trường.
Phân tích rủi ro khi build nhầm production API cho bản test.
🎯 Tư duy: DevOps & release management.

11 Giảm số lần gọi API dư thừa
Phân tích tình huống widget rebuild gây gọi API ID lặp lại.
Đề xuất giải pháp debounce, memoization hoặc caching state.
🎯 Tư duy: tối ưu hiệu năng thực tế.

12 Bảo mật dữ liệu nhạy cảm trên client
Phân tích rủi ro khi lưu token trong SharedPreferences.
So sánh với flutter_secure_storage.
Đề xuất best practice bảo mật mobile app.
🎯 Tư duy: mobile security.

13 Khi Backend trả dữ liệu quá lớn
Phân tích tác động của payload lớn (JSON 5MB).
Đề xuất kỹ thuật nén (gzip), selective field loading, hoặc GraphQL.
🎯 Tư duy: bandwidth optimization.

14 Viết Unit Test cho API Layer
Thiết kế mock API bằng Mockito.
Phân tích lợi ích của test khi backend thay đổi.
So sánh Unit test và Integration test trong Flutter.
🎯 Tư duy: chất lượng phần mềm.

15 Xử lý đồng thời nhiều request phức tạp
Một màn hình cần gọi API theo ID A → sau đó dùng kết quả gọi tiếp API ID B.
Phân tích dependency chain và error propagation.
Thiết kế luồng async tối ưu và dễ maintain.
🎯 Tư duy: async flow & kiến trúc luồng dữ liệu.

PhucPT10@fpt.edu.vn